

AI Day Trader

Long Premium, Short Leash

An autonomous options agent on Alpaca's MCP server.

Alpaca AI Trading Agents Hackathon - Aug 28 to Sep 4, 2026

Bill McCormick - account `PA3VS39Y5LE2` - MIT

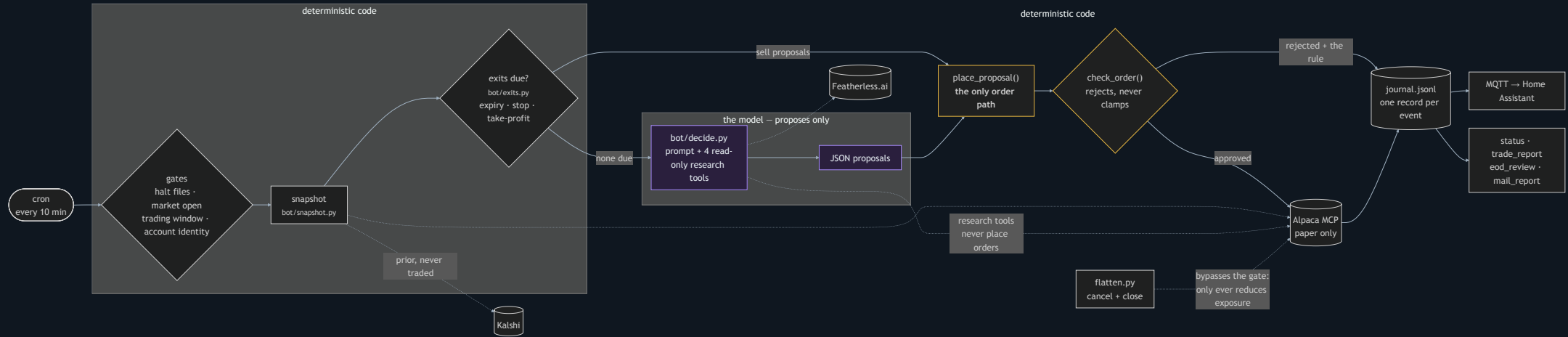
The thesis, in one sentence

Buy defined-risk, short-dated options premium on the most liquid names when an open-source model can *name a reason*; deterministic code sizes every trade, stops it, and closes it before expiry.

The model never touches an order. Open model proposes; code disposes.

Why long premium: the worst case is known before entry, which keeps every guardrail simple and absolute - and it matches what a language model is good at (a directional view from evidence) rather than what it is bad at (managing a position minute to minute).

One cycle, every 10 minutes



Outlined boxes are code only. There is no path from the model to an order.

Alpaca's free options feed has no Greeks

So the agent solves implied volatility from each contract's market price (Black-Scholes, bisection - robust far from the money) and derives delta, gamma, theta and vega on the fly.

```
{"symbol":"SPY260831P00769000","type":"put","strike":769.0,"dte":2,  
  "bid":1.64,"ask":1.67,"last":1.67,  
  "iv":0.0827,"delta":-0.4596,"gamma":0.0843,"theta":-0.4282,"vega":0.226}
```

Solved per contract from a noisy feed - so the prompt tells the model they are rough guides, not to audit them.

A second opinion from a prediction market

Kalshi's daily index-close markets are a crowd-implied distribution of today's close - a second, independent estimate to weigh against what the option chain implies. Handed to the model as a *prior, not a signal*: no code acts on it, so it can only persuade.

- SPY via KXINX; prior close 7,731, implied median 7,712 (-0.24%);
P(above prior close) 0.453, P(up>1%) 0.32, P(down>1%) 0.409; volume 70

The harder half is knowing when to ignore it. A barely-traded market still quotes every bucket, and normalising thirty wide spreads yields a *flat* distribution that looks authoritative. That is the real 08-31 event quoted the evening before: its most likely bucket is a 4.6% *down* move, on 70 contracts. Two gates - minimum volume, and entropy over $\log(\text{buckets})$ - withheld it, and the model saw nothing.

Withheld priors are journalled *with the reason*, so "no second opinion today" is answerable rather than absent. Public API, no key, never traded.

The agent investigates before it acts

```
research: get_bars({'symbol': 'SPY', 'timeframe': '15Min'})          -> 3334 chars
research: get_stock_snapshot({'symbol': 'NVDA'})                    -> 622 chars
research: get_option_snapshot({'symbols': 'NVDA260902P00220000,...'}) -> 1339 chars
research: get_news({'symbols': 'NVDA', 'limit': 5})                 -> 1770 chars
model output: [{"instrument":"option","symbol":"NVDA260902P00220000","side":"buy",
  "qty":10,"order_type":"limit","limit_price":4.65,
  "reason":"NVDA is in a clear intraday downtrend (225 -> 218.6); the 220 put with
    4 DTE, delta -0.58, aligns with the bearish momentum"}]
```

Four read-only tools mapped to Alpaca MCP calls, six calls max, every one journaled. Nothing that orders is reachable.

The leash

Per proposal	whitelist - $\leq$ \$5,000 per position - $\leq$ 4 positions - $\leq$ 10 contracts/order - 1-45 DTE - entries 09:45-15:15 ET
Per cycle, before the model	close on expiry day - stop-loss -40% - take-profit +60%
Per day	2% loss $\rightarrow$ flatten everything and halt - <code>HALT</code> file = global kill switch
End of day	close contracts expiring within a day; hold the rest under the stops (judged on Thursday's close)

```
REJECTED buy 12 SPY260904C00775000: qty 12 exceeds max_contracts_per_order 10
```

Stocks and options are both first-class. Every limit above applies to either one - the dollar cap counts `contracts x 100 x price` or share market value, and the position count is combined. The model states which instrument it wants and why: options when the edge is time-bounded, shares when volatility is expensive or the move is a slow grind.

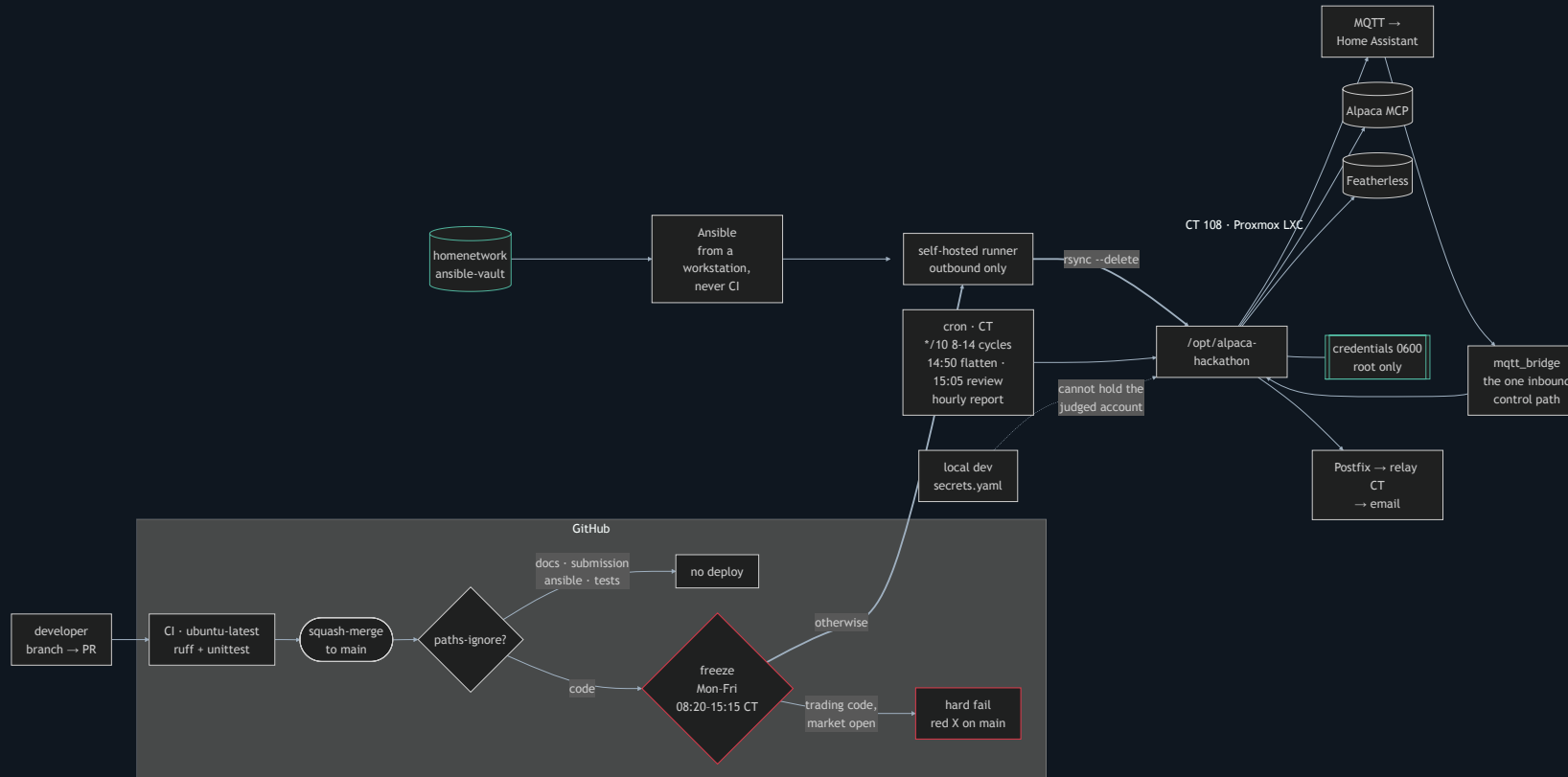
Everything is journaled

One JSONL record per event: `cycle_start`, `config` (hash + active overrides), `decision` (raw output, reasoning, usage, latency, tool calls), `tool_call`, `order_submitted` / `order_rejected` (with the rule), exits, halts.

The journal is what makes the loop evaluable - and what the end-of-day review reads.

It runs on our own hardware, not a laptop

A Proxmox LXC, provisioned by Ansible, driven by *cron*. The runner sits *on* it - a merge deploys in a minute, and the freeze refuses to sync trading code 08:20-15:15 CT. The judging account's keys never leave the box.



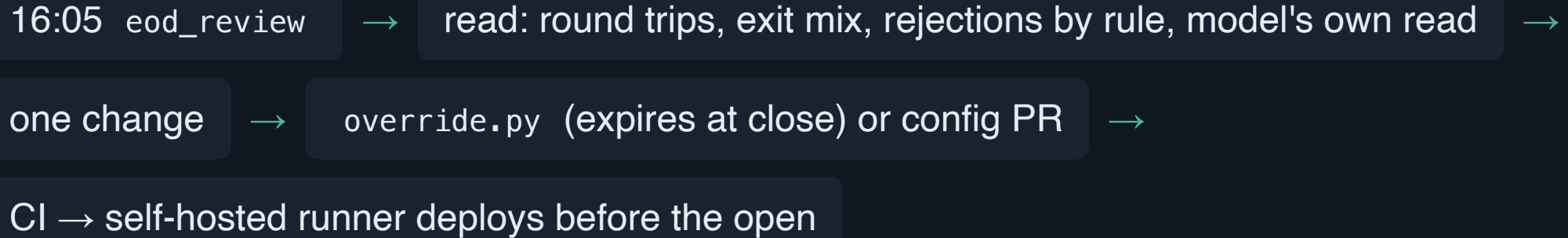
Built to be changed while it is running

A four-day agent is mostly an *iteration speed* problem: the constraint is not writing the strategy, it is changing it without breaking a live account.

506 tests, 1.3s	every one runs with no API keys and no network - clone and get a green suite before you have credentials
Only Docker needed	<code>docker compose run --rm bot</code> - tests, a full dry-run cycle, the docs site
Rehearse any time	<code>--dry-run</code> prints orders instead of sending; <code>--force</code> skips the market gates - a whole cycle is testable at 2am Sunday
Strategy changes without a deploy	<code>override.py</code> - allowlisted knobs, hard risk caps stay git-only, all of it expires at the close

Six of those tests exist because something broke - a report that published on one of four cycle paths, a test that passed only on a laptop. Each incident ends as a test, so the suite is a list of things that can no longer happen quietly.

The daily loop



Two more paper accounts run variant configs against the same live market - the only backtest available in four days. 82 PRs, 506 tests, every merge deployed automatically.

The experiment farm - real A/B, not a backtest

Four days isn't enough for a backtest, and the free feed carries no historical options data anyway. So the A/B runs *live*: three paper accounts on the same box, same ten-minute cadence, same market, different configs - a real experiment rather than a replay.

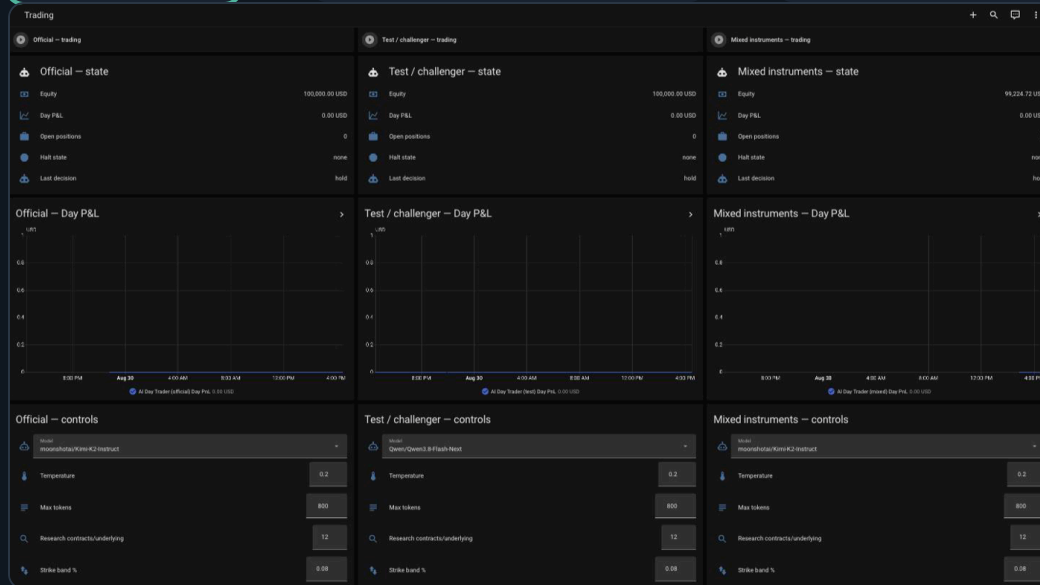
```
# on the trading host, right now - every 10 min, 08:00-14:50 CT
run_cycle.py --account official
run_cycle.py --account test --config config-test.yaml
run_cycle.py --account mixed --config config-variants/mixed.yaml
```

Each is one deliberate change. `test` swaps the model and enables research tools and learning; `mixed` differs by exactly *one key* - stock and options as peers, not options-first - so "which instrument should this agent reach for" is answered by P&L rather than by us.

The same thing runs on a laptop. `docker compose` brings up the bot, the variant farm and the docs site self-contained - no host, no Proxmox, none of our keys. It is how the project is developed, and it would serve just as well to run it.

Home Assistant, over MQTT - fully decoupled

journal.log() → MQTT (fire-and-forget) → HA auto-discovery



One feed, three audiences. **Operator:** kill switch and knobs. **Team:** a read-only copy over Tailscale - HA has no per-entity permissions, so the separation has to be the dashboard. **Phone:** problems only.

A fill sends *no* alert, on purpose: a channel that buzzes on routine trades gets muted, and the halt alert with it. What buzzes is *silence* - no cycle for 25 minutes, the one failure a dashboard cannot show.

Results (fill Thu Sep 3 after the close)

PLACEHOLDER — replace before submitting

Slide 14 — Results

equity curve, Mon open to Thu close

capture Thursday after the close · see [submission/images/README.md](#)

Equity Mon open → Thu close: \$____ → \$____ (____%)

Round trips: ____ - exits: ____ stop / ____ take-profit / ____ expiry / ____ model

Official vs challenger vs mixed: ____

What didn't work: ____

What's next, and thanks

- Promote what the challenger proved: research tools, prediction-market prior, learning loop
- Multi-leg spreads once the gates cover assignment risk
- Home Assistant over MQTT: the light that goes red when the leash pulls

github.com/bill-mccormick-dg/alpaca-hackathon - MIT

Thanks: Alpaca, lablab.ai, Featherless AI